

Inodes, Hard Links and Soft Links

Understanding Linux File References

Prashant Jadav

Fundamentals of Operating System Design (FOSD)

Learning Objectives

- Understand what an inode is.
- Distinguish a filename from an inode.
- Understand hard links and inode sharing.
- Understand symbolic (soft) links and pathname references.
- Compare hard links and soft links experimentally.
- Understand what happens when files are deleted or renamed.

The Basic Idea

In a Unix/Linux filesystem, a filename and the actual file data are not the same thing.

Filename → **Inode** → **Data Blocks**

Key Concept

A directory maps a filename to an inode number. The inode stores metadata and information needed to locate the file's data.

Important

A file can have more than one filename while still having only one inode. This is the basis of hard links.

What is an Inode?

An **inode** is a filesystem data structure containing metadata about a file.

- File type
- File permissions
- Owner and group
- File size
- Timestamps
- Link count
- Pointers to data blocks

Important

The inode normally does **not** contain the filename. The filename is stored in a directory entry.

Filename, Directory Entry and Inode

Conceptually:

Directory Entry	→	Inode
report.txt	→	inode 1257

The inode then identifies the file's metadata and data blocks:

inode 1257 → metadata → data blocks

Key Point

A filename is a name used by the directory. The inode represents the filesystem object.

Viewing Inode Numbers

Use the `-li` option with `ls`:

```
ls -li
```

Example:

```
1257 -rw-r--r-- 1 user user 18 Sep 21 10:30 file1
1261 -rw-r--r-- 1 user user 25 Sep 21 10:31 file2
```

Observation

The first column is the inode number.

You can also use:

```
stat file1
```

What is a Hard Link?

A hard link is another directory entry that refers to the **same inode**.



Core Concept

Two filenames, one inode, one underlying file.

Creating a Hard Link

Create the original file:

```
echo "Hello Linux" > file1
```

Create a hard link:

```
ln file1 file2
```

Check inode numbers:

```
ls -li file1 file2
```

Expected observation:

```
1257 ... file1  
1257 ... file2
```

The inode number is the same.

Hard Link: Link Count

Check the inode information:

```
stat file1  
stat file2
```

After creating the hard link, the inode's link count becomes:

Link count = 2

Conceptually:

file1 → **inode 1257** ← file2

Meaning

The inode remains allocated as long as at least one hard link (directory entry) refers to it.

Deleting a Hard Link

Suppose:

```
ln file1 file2
```

Now remove file1:

```
rm file1
```

What happens?

- The inode is not immediately deleted.
- The link count decreases from 2 to 1.
- file2 still refers to the inode.
- The data remains accessible.

```
cat file2
```

Output:

```
Hello Linux
```

When is a File Actually Removed?

For a regular file, the filesystem can reclaim the inode and data blocks when:

Link count = 0

and no process is still keeping the file open.

Directory entries → **Link count** → **inode** → **data**

Important

`rm` normally removes a directory entry. It does not necessarily erase the data immediately.

What is a Soft Link?

A **soft link**, or symbolic link, is a special filesystem object that stores a pathname referring to another file or directory.

Conceptually:

`softlink` $\longrightarrow$ `"file1"` $\longrightarrow$ **inode of file1** $\longrightarrow$ **data**

Core Concept

A symbolic link points to a **pathname**, not directly to the target's inode.

Creating a Soft Link

Create the original file:

```
echo "Hello Linux" > file1
```

Create a symbolic link:

```
ln -s file1 softlink
```

Check:

```
ls -li file1 softlink
```

The symbolic link has its own inode.
It contains a reference to the pathname:

```
file1
```

Hard Link vs Soft Link: Inode Test

Create both:

```
echo "Hello" > original  
ln original hardlink  
ln -s original softlink
```

Check:

```
ls -li original hardlink softlink
```

Expected concept:

Name	Inode	Relationship
original	1257	Original inode
hardlink	1257	Same inode
softlink	1265	Separate inode

Hard Link: What Happens if Original Name is Deleted?

Initially:

original → **inode 1257** ← hardlink

Delete the original name:

```
rm original
```

The hard link still works:

```
cat hardlink
```

Reason

The hard link directly refers to the same inode. The inode still has a link count of at least 1.

Soft Link: What Happens if Target is Deleted?

Initially:

`softlink` → "original" → **inode 1257**

Delete the target:

```
rm original
```

Now:

`softlink` → "original" → **does not exist**

The symbolic link becomes a **dangling/broken link**.

Demonstration: Hard Link

```
mkdir ~/link-demo
cd ~/link-demo

echo "Operating Systems" > original

ln original hardlink

ls -li original hardlink

cat original
cat hardlink

rm original

cat hardlink
ls -li hardlink
```

Demonstration: Soft Link

```
echo "Operating Systems" > original  
  
ln -s original softlink  
  
ls -li original softlink  
  
cat softlink  
  
rm original  
  
ls -l softlink  
cat softlink
```

Expected Result

After deleting original, softlink becomes a broken symbolic link.

Hard Link vs Soft Link

Feature	Hard Link	Soft Link
Same inode as target	Yes	No
Contains pathname	No	Yes
Own inode	No	Yes
Can link directories	Generally no	Yes
Can cross filesystems	No	Yes
Survives target filename deletion	Yes	No
Can become dangling	No	Yes
Link count of target increases	Yes	No

Why Hard Links Cannot Normally Cross Filesystems

Inode numbers are meaningful within a filesystem.

Therefore:

Hard link $\Rightarrow$ same inode $\Rightarrow$ same filesystem

A hard link cannot normally refer to an inode belonging to another filesystem.

Soft links

A symbolic link stores a pathname, so the target can be located in another filesystem.

Hard Link, Soft Link and Bind Mount

	Hard Link	Soft Link	Bind Mount
Same inode	Yes	No	Accesses same object
Pathname stored	No	Yes	No
Filesystem mount	No	No	Yes
Works with directories	Generally no	Yes	Yes
Cross filesystem	No	Yes	Yes
Main concept	Inode reference	Pathname reference	Mount namespace

Memory Aid

Hard link = same inode

Soft link = pathname pointer

Bind mount = alternate mount path

Important OS Concept: Namespace

A pathname is resolved through the filesystem namespace.
For example:

`/home/user/file.txt`

is resolved through directory entries until the filesystem finds the corresponding inode.

Pathname → Directory Entries → Inode → Data Blocks

Hard links, symbolic links, and mount points modify different parts of this process.

Useful Linux Commands

View inode numbers:

```
ls -li
```

Detailed inode information:

```
stat filename
```

Create hard link:

```
ln source hardlink
```

Create soft link:

```
ln -s source softlink
```

Find filesystem information:

```
df -Th
```

Classroom Questions

- 1 Does an inode normally contain the filename?
- 2 Can two filenames have the same inode?
- 3 What happens to the link count when a hard link is created?
- 4 What happens when one hard link is deleted?
- 5 Why does a symbolic link become broken when its target is deleted?
- 6 Why can symbolic links cross filesystems but hard links normally cannot?
- 7 How is a bind mount different from a hard link?

Key Takeaways

- 1 A directory maps a filename to an inode.
- 2 An inode stores metadata and identifies the file's data blocks.
- 3 A hard link creates another directory entry for the same inode.
- 4 A symbolic link is a separate object containing a pathname.
- 5 Deleting one hard-link name does not necessarily delete the file.
- 6 Deleting a symbolic-link target can create a dangling link.
- 7 Hard links normally cannot cross filesystem boundaries.

Three Concepts to Remember

Inode = filesystem object

Hard link = same inode

Soft link = pathname reference

- Silberschatz, Galvin, Gagne, *Operating System Concepts*, 10th Edition.
- William Stallings, *Operating Systems: Internals and Design Principles*, 9th Edition.
- Linux manual pages: `inode(7)`, `ln(1)`, `stat(1)`, `ls(1)`.